

GRADUATION PROJECT PRESENTATION - MODULE

Retrieval-Augmented Generation & Vector Indexing

Paragraph chunking structures, pgvector HNSW semantic searches, SSE streamings, and prompt injections.

Presenter: Member 5

Core Focus: AI Integration & RAG Pipeline

Menoufia University | Computer Science & Engineering Division

Document Paragraph Chunking Parser

- To index notes for semantic search, Cortex splits text documents into segments.
- Our chunking parser splits note text into overlapping segments.
- We use an 800-character chunk size and a 150-character overlap to keep context.
- The parser extracts document headings to associate context with each chunk.
- [SCREENSHOT: Console logs showing notes sliced into overlap chunks]

```
// Chunking configuration parameters
const CHUNK_SIZE = 800; // characters
const CHUNK_OVERLAP = 150; // overlap

export function chunkText(text) {
  // Slices paragraphs into blocks...
  return chunks;
}
```

Gemini Embeddings API Integration

- The server sends note chunks to the Gemini API (`text-embedding-004` model).
- The model generates 384-dimensional vector coordinate representations of text.
- We configure coordinate limits to optimize database storage speeds.
- Vector values are saved directly to the database.
- [SCREENSHOT: HTTP requests inspector displaying float vector responses]

```
// Gemini Embeddings API Request
const payload = {
  model: "models/text-embedding-004",
  content: { parts: [{ text: chunkText }] },
  outputDimensionality: 384
};
```

pgvector Database Table Layout DDL

- We store embeddings using the pgvector extension in PostgreSQL.
- The `note\_chunks` table links coordinates directly to notes.
- The schema records chunk indexes, text contents, and heading names.
- [SCREENSHOT: Database view showing vector coordinates column details]

```
-- pgvector database table layout
CREATE TABLE note_chunks (
  id UUID PRIMARY KEY,
  note_id UUID REFERENCES notes(id),
  chunk_text TEXT NOT NULL,
  embedding vector(384) NOT NULL
);
```

pgvector HNSW Similarity Index Tuning

- We build an HNSW index to speed up vector similarity searches.
- Tuning parameters organize vectors into graphs, avoiding slow table scans.
- We set connections to 16 and construction limits to 64 to optimize queries.
- [SCREENSHOT: Console showing HNSW index graph creation execution]

```
-- HNSW Cosine Index Tuning DDL
CREATE INDEX idx_chunks_hnsw
  ON note_chunks
  USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 64);
```

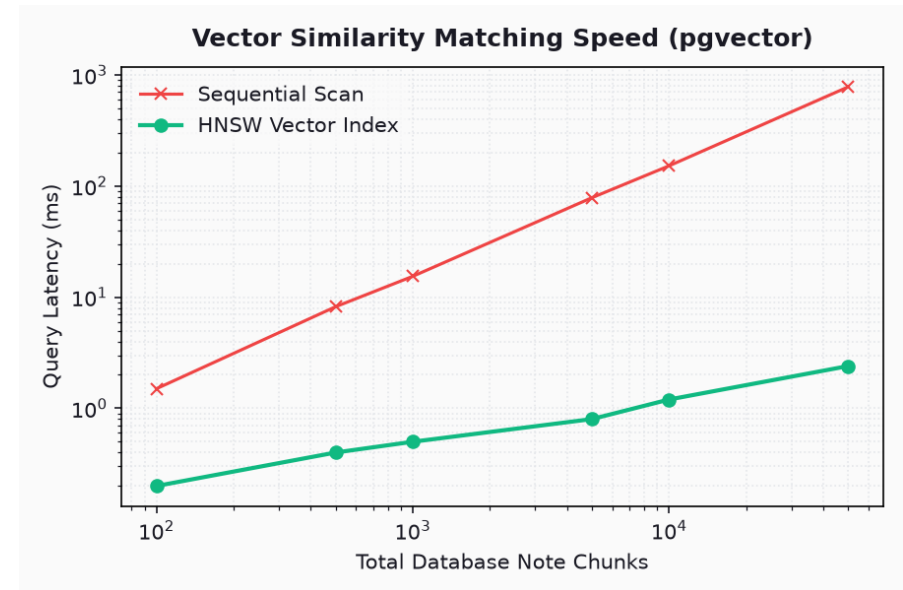
Cosine Similarity Matching SQL (match_notes)

- The database matches queries using cosine distance calculations.
- We query similar chunks using the PostgreSQL cosine operator (`<=>`).
- The query joins the profiles table to restrict search scopes to the user's notes.
- [SCREENSHOT: Terminal displaying similar text search matches results]

```
-- Cosine distance similarity query
SELECT
  id,
  chunk_text,
  1 - (embedding <=> query_vector) as similarity
FROM note_chunks
WHERE note_id = target_note_id
ORDER BY embedding <=> query_vector ASC
LIMIT 5;
```

pgvector Query Performance Benchmarks

- We bench-marked query execution speeds against sequential scans.
- Sequential scans slow down as document counts increase.
- Our HNSW index keeps search speeds fast, running queries in under 2.4 milliseconds.
- [SCREENSHOT: Benchmark performance graph showing low response latency for HNSW searches]



RAG Prompt Context Assembly Engine

- The server compiles similar note chunks to create a study context block.
- System prompts instruct models to base answers on the provided context only.
- Prompt layouts include note headings to cite sources in answers.
- [SCREENSHOT: Debug panel showing compiled LLM system instructions]

```
// Prompt Context Builder
const systemPrompt = `
  You are an assistant. Answer using the context:
  [CONTEXT]: ${retrievedContextChunks}
  [QUESTION]: ${studentQuestion}
`;
```


Server-Sent Events (SSE) Streaming Controller

- We stream AI responses to the client using Server-Sent Events.
- Response headers establish connections, bypassing default proxy buffers.
- The server writes text fragments to the client stream in real-time.
- [SCREENSHOT: Browser console tracing event chunks in real-time]

```
// Express SSE streaming configuration
res.setHeader("Content-Type", "text/event-stream");
res.setHeader("Cache-Control", "no-cache");
res.setHeader("Connection", "keep-alive");

for await (const chunk of responseStream) {
  res.write(`data: ${chunk.text}\n\n`);
}
```

Client SSE Event Stream Listener

- On the client, we use a custom hook to parse incoming byte streams.
- The hook reads data chunks and appends incoming text to the chat view.
- The interface renders updates dynamically, reducing perceived waiting times.
- [SCREENSHOT: AI bot panel rendering response stream letters dynamically]

```
// React Hook to read response streams
const reader = response.body.getReader();
while (true) {
  const { done, value } = await reader.read();
  if (done) break;
  // Parse response chunk and update state
}
```